JAVA FONDAMENTAUX

Présenté par :

Xavier TABUTEAU

Les streams

L'API stream (flux) est introduite dans Java 8, et est utilisée pour traiter des collections d'objet. Un flux est une séquence d'objets qui prend en charge diverse méthodes pouvant être enchaînées pour produire un résultat. Attention, on ne peut parcourir les flux qu'une fois. Si on veut les relire, il faut les recréer.

Les fonctionnalités sont les suivantes :

- Prise en charge d'une collection, d'un tableau ou des entrée / sorties.
- Pas de modification de la structure de données d'origine. Ils fournissent uniquement le résultat selon les méthodes enchaînées.
- Chaque opération intermédiaire est exécutée et renvoie un flux en conséquence. Ces opérations peuvent être enchaînées. Les opérations terminales marquent la fin du flux et renvoient le résultat.

Quelques opérations intermédiaires :

map() : Renvoie le flux constitué des résultats de l'application d'une fonction à ses éléments.
filter() : Renvoie les éléments d'un flux qui correspondent à un prédicat passé en argument.
sorted() : Renvoie un flux trier.
limit(n) : Limite le nombre d'élément retourner en cas de nombre de valeurs infinies.

Les streams

Quelques opérations terminales :

forEach()	: Utilisée pour parcourir chaque élément du flux.
reduce()	: Utilisée pour réduire le flux à une seule valeur. La méthode prendre un opérateur binaire en paramètre.
min()	: Renvoie le plus petit élément du flux.
max()	: Renvoie le plus grand élément du flux.
count()	: Renvoi le nombre d'élément contenu dans le stream
collect()	: Renvoie le résultat des opérations intermédiaires effectuées sur le flux.
collect(Collectors.toList())	: Renvoi le flux sous forme de List.
collect(« type »[]::new)	: Renvoi le flux sous forme de tableau.

Les streams

- **Spécificités des traitements Big Data**

Dans le contexte Big Data, les streams Java prennent un sens particulier. On parle ici de données massives, souvent distribuées et en temps réel.

Voici les points clés :

- Traitement en flux (Stream Processing) : Big Data ne traite pas seulement des données stockées (batch), mais aussi des flux de données continus (IoT, logs, clics web…). Les frameworks Big Data basés sur Java, comme Apache Flink, Apache Kafka Streams, ou Spark Streaming, utilisent un concept proche des streams Java : traitement pipeline avec transformations et agrégations.

- Parallélisme et distribution : Les streams Java peuvent être parallèles (parallelStream()), mais pour Big Data, on parle souvent de distribution sur des clusters.
  Exemple : Spark utilise le concept de RDDs (Resilient Distributed Datasets) qui fonctionne comme des streams distribués, permettant de traiter des téraoctets de données simultanément sur plusieurs machines.

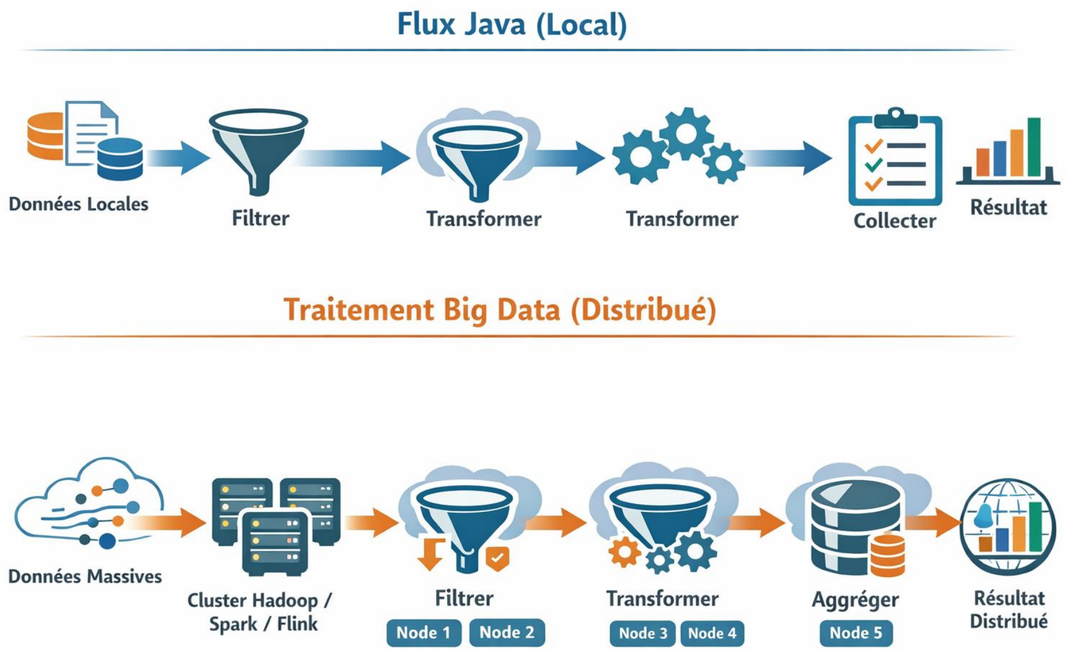
- Opérations “lazy” et immutables : Comme les streams Java, les frameworks Big Data optimisent les calculs avec des opérations paresseuses : rien n’est exécuté tant qu’une action terminale n’est pas appelée. Ceci réduit les passes sur les données et optimise la mémoire et les entrées / sorties.

Les streams

- **Spécificités des traitements Big Data**

- Traitement temps réel vs batch : Batch (Hadoop MapReduce, Spark) : traitement massif mais différé. Temps réel / Stream (Kafka, Flink, Spark Streaming) : traitement au fur et à mesure que les données arrivent. Streams Java donnent la structure de pipeline de transformations qui est conceptuellement similaire à ces frameworks.

- Limitations : Les streams Java standard ne sont pas conçus pour des volumes de données très importants dépassant la mémoire d'une seule machine. Pour le Big Data massif, il faut combiner Java streams avec des frameworks distribués.



JAVA FONDAMENTAUX

Présenté par
Xavier TABUTEAU